

Android XML 原生EditText详解（基础+自定义）

在 Android 传统 XML 开发中，EditText（输入框）是承载用户文本输入的核心控件，广泛用于登录注册、信息填写、搜索框等场景。它继承自 TextView，在其基础上扩展了文本编辑功能。本文从「原生基础用法」到「深度自定义样式与功能」全面拆解，搭配可直接运行的代码示例，帮助开发者快速掌握EditText的使用与定制技巧。

一、核心概念：EditText 是什么？

EditText 作为交互式文本控件，核心特性如下：

- **编辑特性**：支持文本输入、删除、选中、复制粘贴等基础编辑操作，可通过属性限制输入内容（如仅数字、手机号格式）
- **状态反馈**：提供输入状态监听（如文本变化、焦点获取/失去），支持错误提示、输入长度限制等交互反馈
- **样式可定制**：原生样式与系统主题绑定，可通过 XML 属性修改背景、光标、提示文本等，也能通过自定义drawable和布局实现个性化样式
- **输入适配**：支持关联软键盘类型（如数字键盘、邮箱键盘），优化不同场景的输入体验

二、基础用法：原生 EditText（3 步上手）

原生EditText的使用无需复杂配置，核心是「布局添加控件+属性配置+事件监听」，适合简单的文本输入场景。

步骤 1：在 XML 布局中添加 EditText

创建 activity_main.xml，添加常见场景的EditText（如用户名、密码输入框）：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:orientation="vertical"
7      android:padding="20dp"
8      android:background="@color/gray_f5">
```

```
9
10
11 <EditText
12     android:id="@+id/et_username"
13     android:layout_width="match_parent"
14     android:layout_height="50dp"
15     android:hint="请输入用户名"
16     android:inputType="text"
17     android:maxLength="16"
18     android:paddingHorizontal="15dp"
19     android:background="@drawable/et_bg_normal"
20     android:layout_marginBottom="15dp"/>
```

```
21
22
23 <EditText
24     android:id="@+id/et_password"
25     android:layout_width="match_parent"
26     android:layout_height="50dp"
27     android:hint="请输入密码（6-12位）"
28     android:inputType="textPassword"
29     android:maxLength="12"
30     android:minLength="6"
31     android:paddingHorizontal="15dp"
32     android:background="@drawable/et_bg_normal"
33     android:layout_marginBottom="15dp"/>
```

```
34
35
36 <EditText
37     android:id="@+id/et_phone"
38     android:layout_width="match_parent"
39     android:layout_height="50dp"
40     android:hint="请输入手机号"
41     android:inputType="phone"
42     android:maxLength="11"
43     android:paddingHorizontal="15dp"
44     android:background="@drawable/et_bg_normal"
45     android:layout_marginBottom="15dp"/>
```

```
46
47
48 <EditText
49     android:id="@+id/et_remark"
50     android:layout_width="match_parent"
51     android:layout_height="120dp"
52     android:hint="请输入备注（可选）"
53     android:inputType="textMultiLine"
54     android:maxLength="200"
55     android:padding="15dp"
```

```
56         android:background="@drawable/et_bg_normal"
57         android:gravity="top" />
58
59     </LinearLayout>
```

补充：基础背景样式（et_bg_normal.xml）

为输入框添加圆角白色背景，提升视觉效果，在 res/drawable 目录下创建：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <shape xmlns:android="http://schemas.android.com/apk/res/android"
3      android:shape="rectangle">
4      <solid android:color="@color/white"/>
5      <corners android:radius="8dp"/>
6      <stroke android:width="1dp" android:color="@color/gray_e0"/>
7  </shape>
```

在 res/color/color.xml 中添加相关颜色：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="white">#FFFFFF</color>
4      <color name="gray_f5">#F5F5F5</color>
5      <color name="gray_e0">#E0E0E0</color>
6      <color name="blue">#2196F3</color>
7      <color name="red">#F44336</color>
8      <color name="gray_99">#999999</color>
9  </resources>
```

步骤 2：在 Activity 中处理输入事件

在 MainActivity.kt（或 Java）中绑定控件，监听文本变化、焦点状态等核心事件：

Code block

```
1  import android.os.Bundle
2  import android.text.Editable
3  import android.text.TextWatcher
4  import android.view.View
5  import android.view.inputmethod.InputMethodManager
6  import android.widget.EditText
```

```

7  import android.widget.Toast
8  import androidx.appcompat.app.AppCompatActivity
9
10 class MainActivity : AppCompatActivity() {
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13         setContentView(R.layout.activity_main)
14
15         // 绑定控件
16         val etUsername = findViewById<EditText>(R.id.et_username)
17         val etPassword = findViewById<EditText>(R.id.et_password)
18         val etPhone = findViewById<EditText>(R.id.et_phone)
19         val etRemark = findViewById<EditText>(R.id.et_remark)
20
21         // 1. 监听文本变化 (实时响应输入)
22         etUsername.addTextChangedListener(object : TextWatcher {
23             // 文本变化前
24             override fun beforeTextChanged(s: CharSequence?, start: Int,
count: Int, after: Int) {}
25
26             // 文本变化中 (核心)
27             override fun onTextChanged(s: CharSequence?, start: Int, before:
Int, count: Int) {
28                 val input = s.toString().trim()
29                 // 示例: 用户名长度不足3位时提示
30                 if (input.length in 1..2) {
31                     etUsername.error = "用户名至少3位"
32                 } else {
33                     etUsername.error = null // 清除错误提示
34                 }
35             }
36
37             // 文本变化后
38             override fun afterTextChanged(s: Editable?) {}
39         })
40
41         // 2. 监听焦点变化 (如失去焦点时校验手机号)
42         etPhone.onFocusChangeListener = View.OnFocusChangeListener { v,
hasFocus ->
43             if (!hasFocus) { // 失去焦点时校验
44                 val phone = etPhone.text.toString().trim()
45                 if (phone.isNotEmpty() && !phone.matches(Regex("^1[3-
9]\\d{9}$")))) {
46                     etPhone.error = "请输入正确的手机号"
47                     // 强制获取焦点, 让用户重新输入
48                     etPhone.requestFocus()
49                 }

```

```
50         }
51     }
52
53     // 3. 获取输入内容 (如点击登录按钮时)
54     // 模拟登录按钮点击 (实际开发中需在布局添加Button)
55     findViewById<View>(R.id.btn_login).setOnClickListener {
56         val username = etUsername.text.toString().trim()
57         val password = etPassword.text.toString().trim()
58         val phone = etPhone.text.toString().trim()
59         val remark = etRemark.text.toString().trim()
60
61         // 基础校验
62         when {
63             username.isEmpty() -> etUsername.error = "请输入用户名"
64             password.isEmpty() -> etPassword.error = "请输入密码"
65             phone.isEmpty() -> etPhone.error = "请输入手机号"
66             else -> {
67                 // 校验通过, 提交数据
68                 Toast.makeText(this, "提交成功: $username",
69                     Toast.LENGTH_SHORT).show()
70                 // 隐藏软键盘
71                 hideSoftKeyboard(etUsername)
72             }
73         }
74     }
75
76     // 隐藏软键盘的工具方法
77     private fun hideSoftKeyboard(editText: EditText) {
78         val imm = getSystemService(INPUT_METHOD_SERVICE) as InputMethodManager
79         imm.hideSoftInputFromWindow(editText.windowToken, 0)
80     }
81 }
```

步骤 3：核心 XML 属性详解

EditText 的功能通过 XML 属性灵活配置，以下是开发中高频使用的属性分类说明：

1. 输入控制类（核心）

属性名	作用	示例值
android:inputType	指定输入类型（决定软键盘样式和输入限制）	text（普通文本）、 textPassword（密码）、 phone（手机号）、number

		(纯数字)、 textEmailAddress (邮箱)
android:maxLength	最大输入长度 (超过后无法输入)	"11" (手机号)、"16" (用户名)
android:minLength	最小输入长度 (常用于密码限制)	"6" (密码最小6位)
android:maxLines	最大行数 (超过后滚动或截断)	"1" (单行输入, 默认)、"5" (最多5行)
android:lines	固定行数 (无论内容多少都占对应高度)	"3" (固定3行高度)
android:enabled	是否可编辑 (禁用后灰显, 无法输入)	true/false
android:editable	是否允许编辑 (已过时, 推荐使用 enabled)	true/false

2. 视觉样式类

属性名	作用	示例值
android:hint	提示文本 (输入前显示, 输入后消失)	"请输入用户名"
android:hintTextColor	提示文本颜色	"@color/gray_99"
android:textColor	输入文本颜色	"@color/black"
android:textSize	文本大小	"16sp"
android:background	输入框背景 (可自定义 drawable)	"@drawable/et_bg_normal"
android:gravity	文本对齐方式 (多行输入常用 top)	center (居中)、top (顶部对齐)、left (左对齐)
android:padding	内边距 (避免文本贴边)	"15dp"

3. 光标与选择类

属性名	作用	示例值

android:cursorDrawable	自定义光标样式（替换默认竖线）	"@drawable/et_cursor"
android:textCursorDrawabl e	光标颜色（API 29+ 支持，推 荐用 cursorDrawable）	"@color/blue"
android:selectAllOnFocus	获取焦点时是否全选文本	true/false

三、进阶：自定义 EditText 样式（视觉定制）

原生EditText的视觉样式较为基础，实际开发中常需根据UI设计定制背景、光标、提示文本等，核心方案是「通过自定义drawable实现状态切换」，配合XML属性关联。

场景 1：焦点状态切换样式（常用）

需求：输入框未获取焦点时显示灰色边框，获取焦点时显示蓝色边框，提升交互反馈。

步骤 1：创建状态背景选择器（et_bg_selector.xml）

在 res/drawable 目录下创建，通过 selector 区分焦点状态：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <selector xmlns:android="http://schemas.android.com/apk/res/android">
3
4      <item android:state_focused="true">
5          <shape android:shape="rectangle">
6              <solid android:color="@color/white"/>
7              <corners android:radius="8dp"/>
8              <stroke android:width="1.5dp" android:color="@color/blue"/>
9          </shape>
10     </item>
11
12
13     <item android:state_focused="false">
14         <shape android:shape="rectangle">
15             <solid android:color="@color/white"/>
16             <corners android:radius="8dp"/>
17             <stroke android:width="1dp" android:color="@color/gray_e0"/>
18         </shape>
19     </item>
20 </selector>
```

步骤 2：在 EditText 中引用

```

1 <EditText
2     android:id="@+id/et_username"
3     android:layout_width="match_parent"
4     android:layout_height="50dp"
5     android:hint="请输入用户名"
6     android:inputType="text"
7     android:paddingHorizontal="15dp"
8     android:background="@drawable/et_bg_selector" />

```

场景 2：自定义光标样式

需求：将默认光标改为蓝色粗竖线，提升视觉辨识度。

步骤 1：创建光标drawable（et_cursor.xml）

在 res/drawable 目录下创建，用 shape 定义光标形状：

Code block

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <shape xmlns:android="http://schemas.android.com/apk/res/android"
3     android:shape="rectangle">
4     <solid android:color="@color/blue"/>
5     <size android:width="2dp"/>
6 </shape>

```

步骤 2：关联到 EditText

Code block

```

1 <EditText
2     android:id="@+id/et_username"
3     android:layout_width="match_parent"
4     android:layout_height="50dp"
5     android:cursorDrawable="@drawable/et_cursor"
6     android:hint="请输入用户名"
7     android:background="@drawable/et_bg_selector" />

```

场景 3：带图标的输入框（如搜索框）

需求：输入框左侧显示搜索图标，提升场景辨识度，避免单独添加ImageView。

步骤 1：创建带图标的背景（et_bg_with_icon.xml）

用 layer-list 组合图标和背景：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
3
4      <item>
5          <shape android:shape="rectangle">
6              <solid android:color="@color/white"/>
7              <corners android:radius="25dp"/>
8              <stroke android:width="1dp" android:color="@color/gray_e0"/>
9          </shape>
10     </item>
11
12
13     <item
14         android:drawable="@drawable/ic_search"
15         android:left="15dp"
16         android:top="12dp"
17         android:bottom="12dp">
18     </item>
19 </layer-list>
```

步骤 2：调整输入框padding，避免文本覆盖图标

Code block

```
1  <EditText
2      android:id="@+id/et_search"
3      android:layout_width="match_parent"
4      android:layout_height="50dp"
5      android:hint="请输入搜索内容"
6      android:inputType="text"
7      android:background="@drawable/et_bg_with_icon"
8      android:paddingLeft="45dp"
9      android:paddingRight="15dp"
10     android:singleLine="true" />
```

注：ic_search.png 为搜索图标，需放入 res/drawable 目录。

四、高级：自定义 EditText 功能（逻辑扩展）

除视觉样式外，实际开发中常需扩展EditText的功能（如输入格式化、防抖动、密码明文切换），核心方案是「继承 EditText 重写相关方法」或「通过 TextWatcher 实现」。

场景 1：自定义手机号输入框（自动加空格）

需求：输入手机号时自动按「3-4-4」格式添加空格（如138 1234 5678），提升可读性。

步骤 1：创建自定义EditText类（PhoneEditText.kt）

Code block

```
1  import android.content.Context
2  import android.text.Editable
3  import android.text.InputFilter
4  import android.text.InputType
5  import android.text.TextWatcher
6  import android.util.AttributeSet
7  import androidx.appcompat.widget.AppCompatEditText
8
9  class PhoneEditText @JvmOverloads constructor(
10      context: Context,
11      attrs: AttributeSet? = null,
12      defStyleAttr: Int = 0
13  ) : AppCompatEditText(context, attrs, defStyleAttr) {
14
15      init {
16          // 初始化：设置输入类型为手机号，最大长度13（含2个空格）
17          inputType = InputType.TYPE_CLASS_NUMBER
18          filters = arrayOf(InputFilter.LengthFilter(13))
19          addTextChangedListener(PhoneTextWatcher())
20      }
21
22      // 手机号格式化文本监听器
23      private inner class PhoneTextWatcher : TextWatcher {
24          private var isEditing = false // 避免递归调用
25
26          override fun beforeTextChanged(s: CharSequence?, start: Int, count:
Int, after: Int) {}
27
28          override fun onTextChanged(s: CharSequence?, start: Int, before: Int,
count: Int) {}
29
30          override fun afterTextChanged(s: Editable?) {
31              if (isEditing || s == null) return
32              isEditing = true
33
34              val original = s.toString().replace(" ", "") // 去除所有空格
35              val sb = StringBuilder()
36
```

```

37         // 按3-4-4格式添加空格
38         for (i in original.indices) {
39             sb.append(original[i])
40             when (i) {
41                 2 -> sb.append(" ") // 第3位后加空格 (索引2)
42                 6 -> sb.append(" ") // 第7位后加空格 (索引6)
43             }
44         }
45
46         // 设置格式化后的文本，光标移至末尾
47         setText(sb.toString())
48         setSelection(sb.length)
49
50         isEditing = false
51     }
52 }
53
54 // 对外提供获取纯数字手机号的方法 (去除空格)
55 fun getPurePhone(): String {
56     return text.toString().replace(" ", "")
57 }
58 }

```

步骤 2：在 XML 中使用自定义 EditText

Code block

```

1  <com.example.yourpackage.PhoneEditText
2      android:id="@+id/et_phone_custom"
3      android:layout_width="match_parent"
4      android:layout_height="50dp"
5      android:hint="请输入手机号"
6      android:paddingHorizontal="15dp"
7      android:background="@drawable/et_bg_selector"
8      android:layout_marginBottom="15dp"/>

```

步骤 3：在 Activity 中获取纯数字手机号

Code block

```

1  val phoneEditText = findViewById<PhoneEditText>(R.id.et_phone_custom)
2  val purePhone = phoneEditText.getPurePhone() // 获取无空格的手机号，用于接口提交

```

场景 2：带密码明文切换的输入框

需求：密码输入框右侧添加「显示/隐藏」按钮，点击切换密码明文/密文状态，提升用户体验。

步骤 1：创建自定义布局（et_password_with_toggle.xml）

用 LinearLayout 包裹 EditText 和切换按钮：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="50dp"
6      android:orientation="horizontal"
7      android:background="@drawable/et_bg_selector"
8      android:gravity="center_vertical">
9
10     <EditText
11         android:id="@+id/et_password_inner"
12         android:layout_width="0dp"
13         android:layout_height="match_parent"
14         android:layout_weight="1"
15         android:hint="请输入密码"
16         android:inputType="textPassword"
17         android:maxLength="12"
18         android:paddingHorizontal="15dp"
19         android:background="@null" />
20
21     <ImageView
22         android:id="@+id/iv_toggle"
23         android:layout_width="24dp"
24         android:layout_height="24dp"
25         android:src="@drawable/ic_password_hide"
26         android:layout_marginRight="15dp"
27         android:clickable="true"
28         android:focusable="true"/>
29
30 </LinearLayout>
```

步骤 2：创建自定义EditText容器类（PasswordEditText.kt）

Code block

```
1  import android.content.Context
2  import android.util.AttributeSet
3  import android.view.LayoutInflater
4  import android.view.View
```

```

5  import android.widget.EditText
6  import android.widget.ImageView
7  import android.widget.LinearLayout
8  import androidx.appcompat.widget.AppCompatEditText
9  import com.example.yourpackage.R // 替换为你的包名
10
11  class PasswordEditText @JvmOverloads constructor(
12      context: Context,
13      attrs: AttributeSet? = null,
14      defStyleAttr: Int = 0
15  ) : LinearLayout(context, attrs, defStyleAttr) {
16
17      private val etPassword: EditText
18      private val ivToggle: ImageView
19      private var isPasswordVisible = false // 密码是否可见
20
21      init {
22          // 加载自定义布局
23          LayoutInflater.from(context).inflate(R.layout.et_password_with_toggle,
24 this, true)
25          // 绑定控件
26          etPassword = findViewById(R.id.et_password_inner)
27          ivToggle = findViewById(R.id.iv_toggle)
28
29          // 切换按钮点击事件
30          ivToggle.setOnClickListener {
31              togglePasswordVisibility()
32          }
33
34          // 继承EditText的属性 (如hint、textSize等)
35          attrs?.let {
36              val typedArray = context.obtainStyledAttributes(it,
37 R.styleable.PasswordEditText)
38              val hint =
39 typedArray.getString(R.styleable.PasswordEditText_android_hint)
40              val textSize =
41 typedArray.getDimensionPixelSize(R.styleable.PasswordEditText_android_textSize,
42 16)
43
44              etPassword.hint = hint
45              etPassword.textSize = textSize.toFloat() /
resources.displayMetrics.scaledDensity
46              typedArray.recycle()
47          }
48      }
49
50      // 切换密码可见性
51      private fun togglePasswordVisibility() {

```

```

46         isPasswordVisible = !isPasswordVisible
47         // 修改输入类型: 明文 (textVisiblePassword) / 密文 (textPassword)
48         val inputType = if (isPasswordVisible) {
49             InputType.TYPE_CLASS_TEXT or
InputType.TYPE_TEXT_VARIATION_VISIBLE_PASSWORD
50         } else {
51             InputType.TYPE_CLASS_TEXT or InputType.TYPE_TEXT_VARIATION_PASSWORD
52         }
53         etPassword.inputType = inputType
54         // 保持光标在末尾
55         etPassword.setSelection(etPassword.text.length)
56         // 更新图标
57         ivToggle.setImageResource(
58             if (isPasswordVisible) R.drawable.ic_password_show else
R.drawable.ic_password_hide
59         )
60     }
61
62     // 对外提供EditText的常用方法
63     fun getText(): String = etPassword.text.toString().trim()
64     fun setError(error: CharSequence?) = etPassword.error = error
65     fun requestFocus() = etPassword.requestFocus()
66 }

```

步骤 3: 在 attrs.xml 中定义自定义属性（可选）

为了让自定义控件支持XML属性配置，在 res/values/attrs.xml 中添加：

Code block

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <declare-styleable name="PasswordEditText">
4          <attr name="android:hint"/>
5          <attr name="android:textSize"/>
6          <attr name="android:maxLength"/>
7      </declare-styleable>
8  </resources>

```

步骤 4: 在 XML 中使用

Code block

```

1  <com.example.yourpackage.PasswordEditText
2      android:id="@+id/et_password_custom"
3      android:layout_width="match_parent"

```

```
4      android:layout_height="50dp"
5      android:hint="请输入密码"
6      android:textSize="16sp"
7      android:maxLength="12"
8      android:layout_marginBottom="15dp"/>
```

五、关键知识点总结（必记）

知识点	核心内容
输入控制核心	通过 android:inputType 控制软键盘类型和输入格式，减少无效输入
事件监听重点	TextWatcher 监听文本实时变化， onFocusChangeListener 监听焦点状态（用于失去焦点校验）
样式自定义思路	用 selector 实现状态切换（焦点/普通），layer-list 组合背景与图标， cursorDrawable 自定义光标
功能扩展方案	简单需求用 TextWatcher，复杂需求继承 EditText 重写方法，或通过组合布局实现（如密码切换）
性能优化	避免在 TextWatcher 中频繁操作 UI；使用 InputFilter 限制输入长度，减少后续校验压力

六、常见问题排查

1. 输入框无法获取焦点

原因：父布局设置了 android:clickable="true" 或 android:focusable="true" 抢占焦点；EditText 被设置为 android:enabled="false" 或 android:focusable="false"。

解决：移除父布局的 focusable 属性；确保 EditText 的 enabled 和 focusable 为 true。

2. 软键盘遮挡输入框

原因：Activity 未配置软键盘适配属性，输入框在屏幕底部时被遮挡。

解决：在 AndroidManifest.xml 中对应 Activity 添加属性

```
android:windowSoftInputMode="adjustResize"，让布局自动上移。
```

3. 文本监听器出现递归调用

原因：在 TextWatcher 的 afterTextChanged 中修改 EditText 的文本，会再次触发文本变化事件，导致递归。

解决：添加标志位（如 isEditing），修改文本前设为 true，修改后设为 false，在事件开始时判断标志位跳过处理。

4. 自定义控件属性不生效

原因：未在自定义控件中解析 attrs 属性；属性名与系统属性冲突；未调用 typedArray.recycle() 释放资源。

解决：在 init 中解析 obtainStyledAttributes 获取属性；使用自定义命名空间区分属性；解析后必须调用 recycle()。

本文覆盖了 EditText 从基础使用到深度定制的全场景需求，开发者可根据实际业务选择对应的实现方案。如果需要实现更特殊的功能（如身份证号输入格式化、输入内容防抖动提交），可以随时补充需求进行扩展！

（注：文档部分内容可能由 AI 生成）